탁찬홍 포트폴리오

About Me

대용량 트래픽 환경에서 **성능·동시성·정합성 문제를 직접 설계하고 검증**한 백엔드 개발자입니다.

- **[조회 성능 동시성]** 300만 건 경매 조회 **5,115ms → 31ms** (99.4%) / Redisson 분산락으로 DB Connection Pool 독점 해소 → 조회 응답 **1,003ms → 44ms** (95.6%)
- **[정합성]** TransactionalEventListener + Claim Check 패턴으로 **Deadlock 원천 제거 및 이벤트 유실 0건** 달성 — 100개 스레드 동시성 테스트 검증
- **[외부 검증]** LLM 기반 Spring 서버 진단 MCP **카카오 PlayMCP 공식 심사 통과** — 1인 개발, Bulk Insert로 TPS 68.9% 향상 / Sidecar 기반 보안 로그 수집 아키텍처 설계

Tool & Skill.

Backend

- Java / Spring Boot
- Spring Data JPA, QueryDSL
- 대용량 조회 최적화 및 Pagination 전략 설계
- JUnit 5, Mockito 를 활용한 단위/통합 테스트 및 동시성 이슈 검증

Infrastructure & DevOps

- Docker
- AWS EC2, RDS, S3
- GitHub Actions

Database

- MySQL
- Covering Index / Deferred Join 기반 성능 튜닝
- Deadlock 분석 및 트랜잭션 구조 개선
- Redis (Redisson 분산 락)

Architecture & Design Pattern

- Claim Check 기반 이벤트 정합성 보장
- 멱등성 Key 기반 중복 처리 방지
- Sidecar 패턴 기반 로그 수집 아키텍처

Project.

1. 경매 플랫폼 'NAFAL' (해커톤 최우수상 및 아키텍처 리팩토링)

Information

개발 기간: 2025/08/12 ~ 2025/08/25 (해커톤) / 2025.11 ~ 2026.03

팀 구성: 5인 (해커톤) / 개인(리팩토링)

담당 업무: 경매 조회 로직 및 백오피스 개발, 프론트 인력 이탈 대응으로 UI/API 연동 전담

기술 블로그: https://velog.io/@takch02/series/%EB%A6%AC%ED%8E%99%ED%86%A0%EB%A7%81

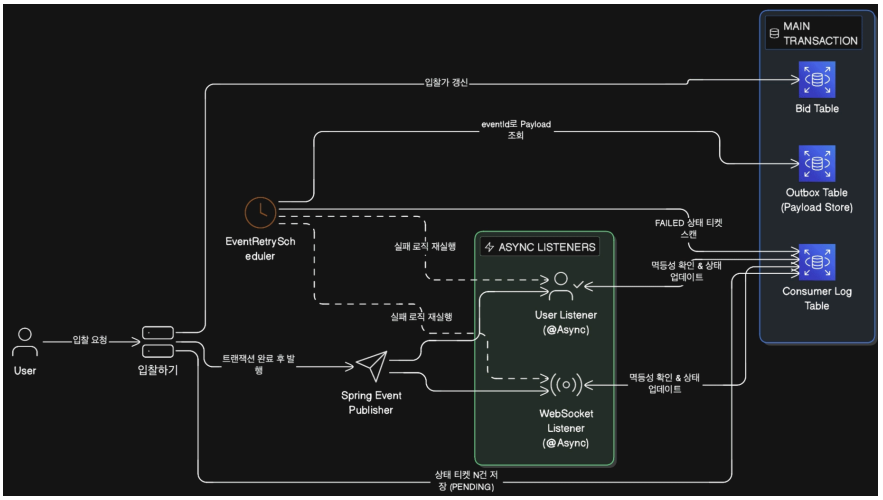
Github : https://github.com/Takch02/blabus_session2_item1

1. 프로젝트 개요 및 해커톤 성과

- 업사이클링 제품을 거래하는 역경매 플랫폼으로, 블레이버스 MVP 해커톤 시즌 2에서 최우수상을 수상
- **위기 대응 및 해커톤 완수:** 프론트엔드 인력 이탈 상황에서 UI/API 연동을 자발적으로 전담하여 팀의 개발 공백을 메우고 **최우수상 수상을 견인**
- **아키텍처 고도화:** 대회 종료 후, 해커톤에서 체감한 구조적 한계를 바탕으로 대규모 운영 환경을 가정한 **동시성 제어 및 조회 성능 최적화 리팩토링**을 개인적으로 진행

2. 핵심 트러블슈팅 및 아키텍처 최적화 (리팩토링)

1. **Outbox & Consumer Log 패턴 기반 Deadlock 해결 및 이벤트 정합성 보장 아키텍처 설계**



문제:

- 입찰과 User 정보 수정 트랜잭션이 서로 다른 순서로 교차 Lock(Auction↔User)을 획득하며 **DeadLock 발생**
- 비동기 이벤트로 분리 시 이벤트 부분 실패 발생

해결:

- User 업데이트 로직을 **이벤트 기반으로 분리**하여 트랜잭션 간 Lock 충돌 제거 (EventListener 활용)
- 이벤트 데이터 본체는 `Outbox`에 단일 저장하고, 실행 상태는 `Consumer Log`로 분리하는 **Claim Check 패턴** 적용
- 수신 측에서 Consumer Log 상태를 사전 검증하여 **중복 실행 방어 (멱등성 확보)**

결과:

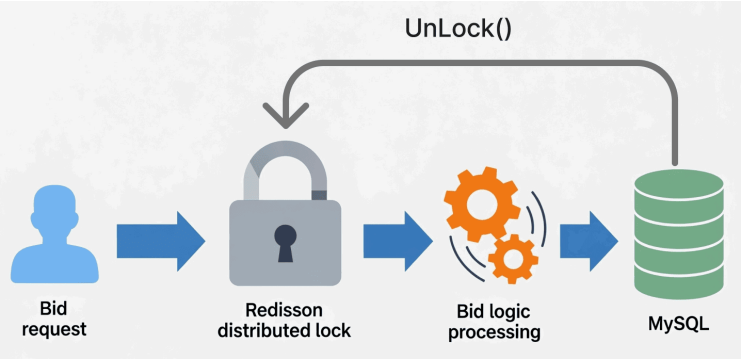
- **Deadlock 원천 차단: 100개 스레드 규모의 동시성 부하 테스트**를 통해 트랜잭션 분리에 따른 **DeadLock 미발생 검증**
- **결과적 일관성 100% 증명:** Mockito를 활용해 **비동기 이벤트의 의도적 부분 실패 상황을 재현** 후 Consumer Log 스케줄러의 개입을 통한 **자동 복구 및 멱등성 유지 로직 검증**

2. **Redisson 분산 락 도입을 통한 DB Connection Pool 고갈 해결**

문제:

- 비관 락 기반 입찰 로직이 DB Connection을 점유하며 대기, 경매 조회 등 타 서비스 요청에 필요한 Connection이 고갈되는 문제 발생
- 경매 조회(타 서비스 요청) 응답시간 1,003ms, TPS 미달

해결: Redisson 분산 락(Pub/Sub 방식)으로 Lock 관리를 DB에서 분리



- 낙관 락 + 재시도는 경매 특성상 재시도 자체가 무의미하고 $O(N^2)$ 재시도 폭발 위험으로 배제 후 Redisson 분산 락 도입
- DB Connection 1개로 입찰 처리, 나머지 Connection은 타 서비스에 온전히 할당
- TTL 5초 설정으로 장시간 대기 대신 429 응답 처리

검증: 단일 입찰 테스트만으로는 문제가 드러나지 않아, 실제 운영 환경을 가정한 통합 시나리오 설계

- 시나리오 1: 150명 동시 입찰 (200건 / 10초)
- 시나리오 2: 50명 경매 조회 (10초 지속)
- Connection Pool 30개, 비동기 Task Pool 15개 제한 하여 운영 환경의 자원 제약을 의도적으로 재현

결과 : 로컬 환경 부하 테스트(K6) 기준 / MacBook M2 Air 16GB / DB-App 동일 머신

- 입찰 200건 + 경매 조회 2,000건 = **전체 요청 100% 성공**

	개선 전(비관 락)	개선 후(분산 락)	개선율
조회 응답	1,003ms	44ms	95.6% 개선
전체 TPS	1,804	2,200	22% 향상

3. **Covering Index와 Deferred Join을 통한 대용량 데이터 조회 최적화**

   문제:

   - 대용량 경매 데이터(300만 건) 페이징 조회 병목 발생 (응답 5,115ms)
   - `JPA Page`의 무거운 `COUNT` 쿼리 및 `OFFSET` 스캔으로 인한 풀 테이블 접근
   - DB Connection Pool 고갈 및 전체 시스템 응답 지연

   해결 :

   - **page → slice 전환:** 무거운 COUNT 쿼리를 제거하여 불필요한 전체 행 탐색 부하 해소
   - **Covering Index & Deferred Join : 기존의 `findAll()` 으로 인한 데이터베이스 과부하를 막기 위해, Index 로 식별자를 먼저 찾는** Covering Index와 Deferred Join 구조를 채택하여 조회 성능 극대화
   - 300만개의 더미 데이터를 적재하여 부하 상황을 가정하고 테스트

   결과: 로컬 환경 부하 테스트(K6) 기준 / MacBook M2 Air 16GB / DB-App 동일 머신

	개선 전	개선 후	개선율
경매 조회 응답	5,115ms	31ms	168배 개선
에러율	12.5%	0%	100% 개선

2. LLM 기반 Spring 서버 진단 MCP (Server Doctor MCP)

Information

개발 기간: 2026.01 ~ 현재

팀 구성: 개인

기술 블로그 : https://velog.io/@takch02/series/MCP

Github : https://github.com/Takch02/server-monitoring-MCP-server

배포 : https://playmcp.kakao.com/mcp/412

1. 프로젝트 배경

> 초기 서비스나 사이드 프로젝트에서는 **서버 상태 확인과 장애 분석 과정이 대부분 수동으로 이루어지는 문제를 발견했습니다.**
>
> **기존 문제점**
>
> - 에러 로그를 수동으로 수집 후 분석
> - 모니터링 도구(DataDog 등)의 높은 도입 비용
> - 서버 장애 발생 시 원인 파악에 긴 시간 소요
>
> 이 문제를 해결하기 위해 **LLM 기반 서버 진단 플랫폼(Server Doctor MCP)** 을 설계했습니다.

2. 시스템 목표

1. 서버 상태 진단 자동화

[기존]

서버 접속 → 로그 조회 → 수동 분석

[개선]

LLM 진단 요청 → Tool 호출 → 자동 분석

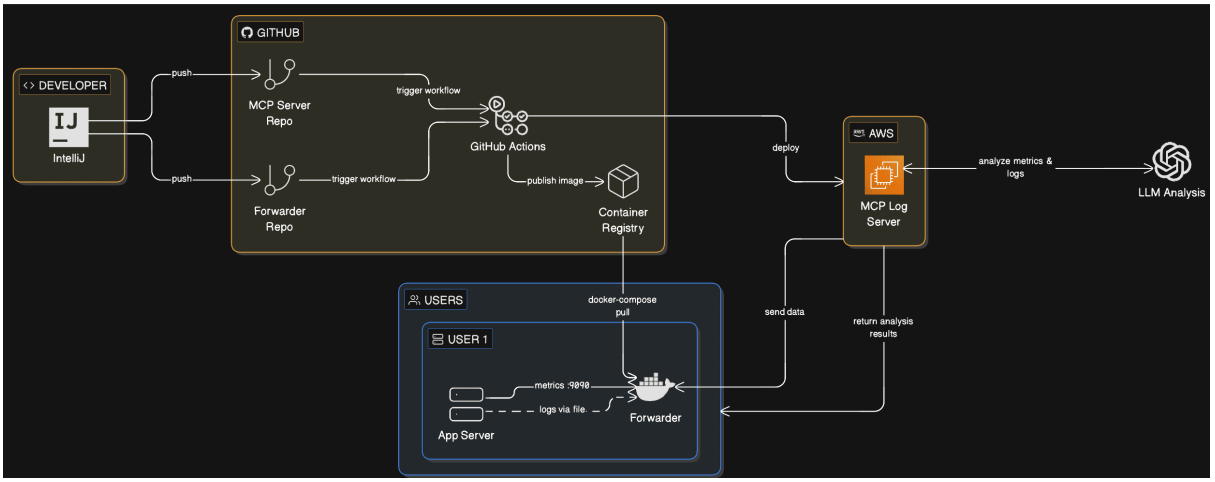
2. 안전한 로그 수집 구조 구축

외부에서 서버 데이터를 직접 조회하는 방식 대신 **Sidecar 기반 Push 방식 로그 수집 구조**를 설계했습니다.

이를 통해

- 외부 포트 노출 제거
- 인증 정보 노출 방지
- 서버와 독립적인 로그 수집 구조 확보

3. 아키텍처



- **Sidecar 패턴 기반 로그 수집 구조 설계**

  문제: Log, Metrics를 수집하기 위해 외부 포트(9090)를 개방하는 과정에서 **심각한 보안 위험이 존재**

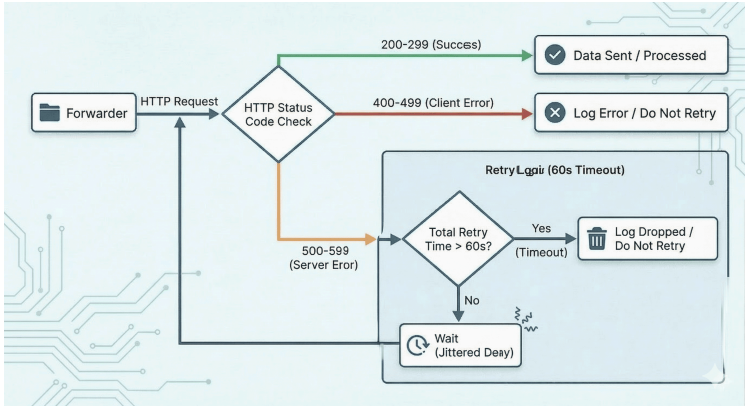
  해결: 사용자 서버 내부에 **Forwarder를 Sidecar 형태로 배치**하여 로그를 Push 방식으로 전송하도록 구조 리팩토링

  결과:

  - 외부 포트 노출을 막으며 Log, Metrics 유출 방지
  - 기존 서버와 독립적인 로그 수집 구조 확보

4. 트러블 슈팅

1. **Queue 와 Jitter을 활용한 로그 유실 및 연쇄 장애 방어**



문제: 통신 장애 시 무한 재시도로 인해 두 가지 구조적 문제 발생

- **리소스 고갈:** 재시도가 집중되며 사용자 서버 메모리 점유 지속 증가, OOM 위험
- **로그 유실:** 재시도 처리에 묶여 신규 로그가 전송되지 못함

해결:

- 재시도 간격에 **Jitter**를 부여해 타겟 서버의 부하 분산
- **TTL 1분** 적용으로 오래된 로그를 폐기해 연쇄 장애 차단
- 최대 300건 제한의 메모리 Queue를 두어 호스트 메인 서버의 메모리 고갈 방어

장애 시나리오 검증

- 로그 수집 서버 중단 시 → 1,000건 Queue 보관 → 복구 후 전량 전송, **유실 0건 확인**

2. **멱등성 키와 DB 제약조건을 활용한 중복 검증 오버헤드 제거**

   문제:

   - 네트워크 재전송 시 로그 중복 발생
   - 무거운 텍스트 대조 비용 및 애플리케이션 `try-catch` 예외 처리 오버헤드 발생

   해결:

   - 메타데이터(서버명, 시간, 메시지) 기반 **SHA-1 멱등성 키** 생성
   - DB 레벨의 **UNIQUE 인덱스(O(logN))** 와 `ON DUPLICATE KEY UPDATE` 활용

   결과: 애플리케이션의 Stack Trace 생성 오버헤드 없이 안전하게 중복 데이터 무시 및 성능 최적화

3. **Bulk Insert와 재시도 파이프라인으로 Deadlock 방어 및 TPS 68.9% 개선**

   문제: 대량 재시도 시 단건 INSERT로 인한 네트워크 병목 및 중복 키 유입 시 **S-Lock → X-Lock 승격**으로 인한 DeadLock 발생

   해결:

   - `JdbcTemplate` 기반 **Bulk Insert** 도입 및 메모리 내 사전 정렬로 인덱스 Page Split 최소화
   - 구조적 데드락을 애플리케이션 레벨에서 방어하기 위해 **Spring** `@Retryable` 적용 (0.2초 후 재시도)

   결과: 로컬 환경 부하 테스트(K6) 기준 / MacBook M2 Air 16GB / DB-App 동일 머신

	개선 전	개선 후	개선율
DB 저장 응답	249.32ms	86.45ms	65.3% 개선
전체 TPS	594.8	1,004.5	68.9% 향상

   - 데드락 에러율 **1% → 0%** 달성

프로젝트 종합 성과

트러블슈팅 파이프라인 자동화

- 서버 SSH 접속 -> 로그 탐색 -> 복사 -> LLM 프롬프트 입력으로 이어지던 파편화된 수동 에러 분석 과정을 **LLM 호출 한 번으로 통합**
- 장애 인지부터 원인 파악까지의 오버헤드를 획기적으로 제거완료

3. 인터랙티브 웹을 위한 3D 데이터 서빙 파이프라인 구축

Information

개발 기간: 2026/01/31 ~ 2026/02/12 (해커톤)

팀 구성: 6인

담당 업무 / 기여도: API 서빙 파이프라인 구축, 네트워크 페이로드 최적화

기술 블로그:
https://velog.io/@takch02/series/%EB%B8%94%EB%A0%88%EC%9D%B4%EB%B2%84%EC%8A%A4-%ED%95%B4%EC%BB%A4%ED%86%A4-%EC%8B%9C%EC%A6%8C4

Github : https://github.com/Takch02/3D_project_backend/tree/main/dosa_backend,

1. 프로젝트 개요 및 접근 방식

시간 제약 하의 기술 습득 및 아키텍처 설계

> 해커톤 일정(10일) 안에 Blender를 처음 접한 상태에서, GLB 조립과 좌표 추출에 필요한 부분만 선택적으로 학습하여 Blender → Python → DB → React로 이어지는 3D 데이터 서빙 파이프라인을 설계하고 실제 3D Viewer에 구현까지 완료했습니다.
>
> 백엔드 역할 및 Network / JS Runtime을 포함한 전체 시스템 성능 분석 및 최적화를 담당했습니다.

2. 핵심 트러블 슈팅 및 아키텍처

[아키텍처] 3D Asset과 공간 데이터 분리 설계 및 파이프라인 구축

문제: 초기 제공된 3D Asset이 조립되지 않은 부품 상태로, 브라우저 렌더링에 필요한 위치 좌표와 회전값 메타데이터가 부재했음

해결 :

- **데이터 생성:** Blender를 활용해 3D Asset을 가조립하여 공간상의 기준 좌표 및 회전값 생성
- **데이터 추출 및 저장:** Python 스크립트를 통해 좌표(X,Y,Z) / 회전 데이터를 추출한 뒤 DB에 정규화하여 저장

결과: 3D 부품 위치 변경 시 DB 좌표 값 수정만으로 즉시 화면에 반영되는 동적 렌더링 구조 구축

[네트워크 최적화] 환경광(HDR) 경량화로 초기 화면 출력 지연 해결

문제: 3D 환경광(HDR) 구현을 위한 고해상도 메타데이터(1.2MB)로 인해, Fast 4G 환경 기준 초기 API 리소스 다운로드 병목(약 4,000ms 지연) 발생 확인

해결:

- Chrome DevTools Network 분석을 통해 병목 구간을 식별
- 반사광 계산에 필요한 환경맵 특성 및 클라이언트 해상도를 고려하여 리소스를 1024×512에서 256×128로 리사이징 및 압축 서빙

결과:

- 이미지 용량 1.2MB → **120KB** 감소 (약 90% 개선)
- HDR 다운로드 시간 4,000ms → **300ms** 대폭 단축

RESUME.

🖊️ **학력 :** (2021.02 ~ 현재) 경북대학교 건설방재공학과 / 글로벌소프트웨어융합학과(복수전공) 재학
🖊️ **자격증 :** (2025.12) SQL 개발자 (SQLD)
🖊️ **수상 :** (2025.08) 블레이버스 MVP 해커톤 시즌 2 최우수상
🖊️ **교육 :** (2025.04 ~ 2025.06) 오픈소스 컨트리뷰션 체험형 1차
🖊️ **기술 블로그 :** https://velog.io/@takch02/posts
🖊️ **Github :** https://github.com/Takch02